Commission calculation

Study guide 3: commission/services.py, the broker commission rules in plain pandas

What this guide covers

`commission/services.py` (97 lines): every business rule of the broker commission report. It has no Django imports, so it's plain pandas you can test on tiny tables and call from a notebook.

How to use it

- Follow the worked example in Part 2 by hand first. If you can predict every number, the code will read easily.
- Rebuild the file in Part 3's order, checking each function against the worked example.
- Run the checkpoints in Part 5; they were extracted from this guide and run against the project.

Part 1. Think before you write code

The job

commission/services.py
For one month: each referred client's loss, profit, NBP, net result and own commission; then each broker's totals and brokerage under the MIS rules; plus four headline numbers for the page.

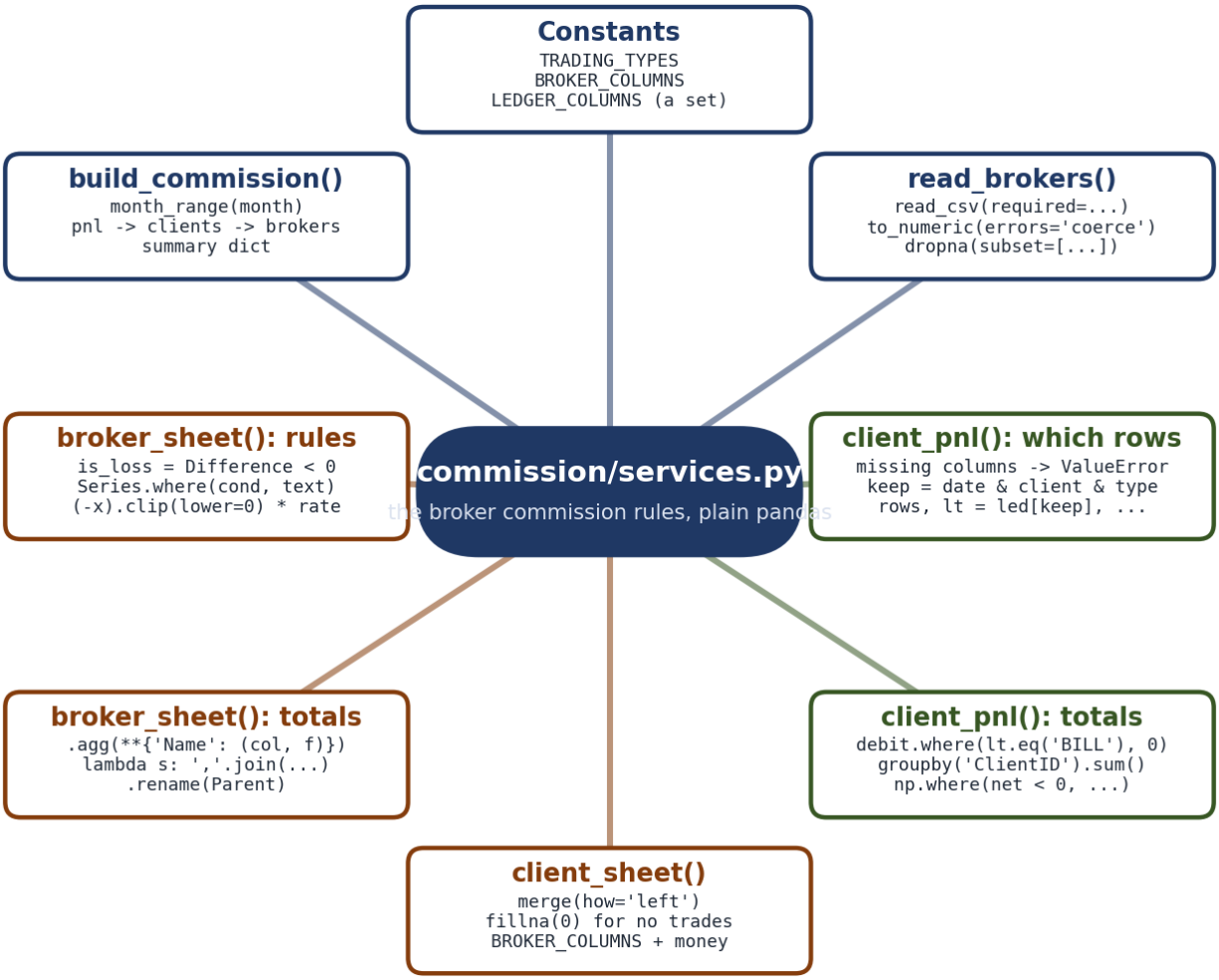
The contract

```
def build_commission(led, brokers, month, rate=0.05) -> dict:
    """The whole report for one month: both sheets plus headline numbers."""
    ...
    return {"clients": <DataFrame>, "brokers": <DataFrame>,
            "summary": {"brokers": int, "clients": int, "loss_brokers": int, "total_brokerage": float}}
```

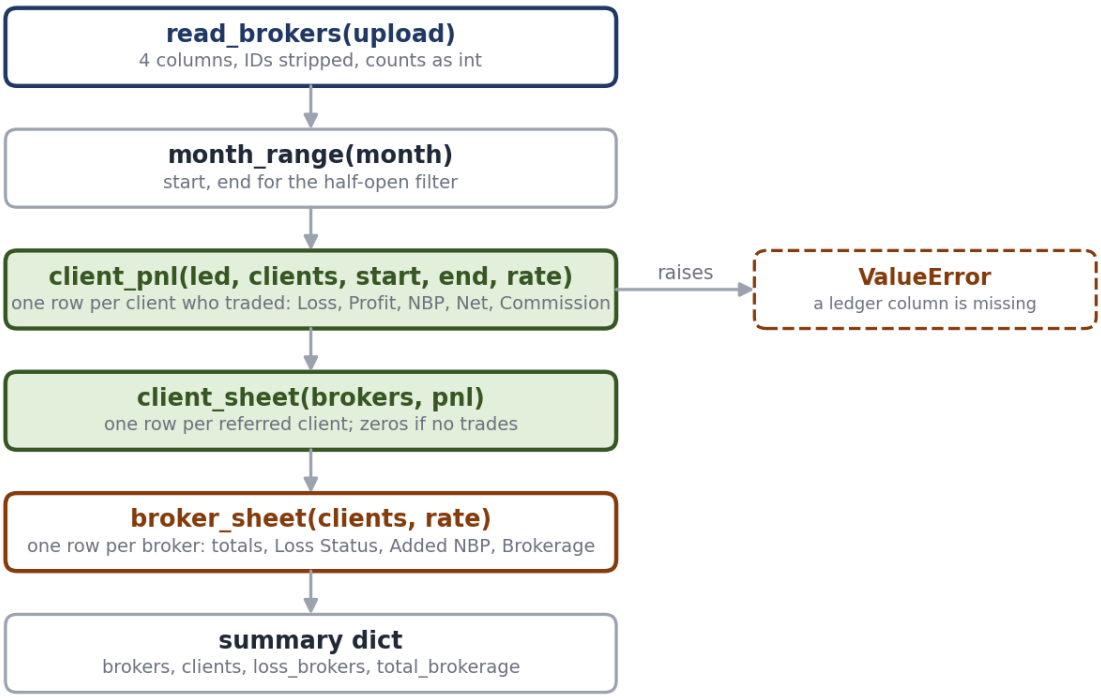
Facts that shape the design

Fact	So the code...	Piece
The ledger may hold messy types ('bill ') and blank amounts	cleans types, treats blanks as 0	`str.strip().str.upper(), fillna(0)`
Only BILL and NBP rows matter	filters both types in one step	`ledger_type.isin(TRADING_TYPES)`
Some referred clients never trade	still lists them, with zeros	`left merge + fillna(0)`
Brokerage is on the broker's combined loss	totals per broker before the rule	`broker_sheet() groupby, then rules`
NBP can cover the whole loss	never pays on a gain	`(-added_nbp).clip(lower=0)`
The MIS sheet mixes numbers and words in one column	keeps numbers numeric	`Series.where(cond, 'TEXT')`
The web, tests and notebooks all need these rules	no Django imports at all	`plain functions, plain pandas`

The whole file as a mindmap



Part 2. The big picture, with a worked example



A worked example you can check by hand

Three brokers, four referred clients, August 2026. Client B's 500 loss is on 1 Sep, so it's outside the month; client C never traded. Note the messy `'bill '` and the blank Debit.

ClientID	CreatedAt	LedgerType	Debit	Credit
A	2026-08-02	`BILL`	100	0
A	2026-08-03	`bill`	(blank)	30
A	2026-08-04	`NEGATIVE BALANCE PROTECTION`	0	10
B	2026-08-05	`BILL`	0	80
B	2026-09-01	`BILL`	500	0
D	2026-08-06	`BILL`	400	0

Brokers: X (P1) referred A and B; Y (P2) referred C; Z (P3) referred D. Rate 5%.

Step by step through the functions:

Client	Loss	Profit	NBP	Net_Profit	Commission
A	100	30	10	-60	3.00 (5% of 60)
B	0	80	0	80	0
C (no trades)	0	0	0	0	0
D	400	0	0	-400	20.00

Broker	Profit	Loss	Difference	NBP	Loss Status	Added NBP	Brokerage
X (A, B)	110	100	10	10	NO LOSS	NOT ELIGIBLE...	NO BROKERAGE
Y (C)	0	0	0	0	NO LOSS	NOT ELIGIBLE...	NO BROKERAGE
Z (D)	0	400	-400	0	LOSS	-400	20.00

The point to notice
Client A has a 3.00 commission on the Clients sheet, yet broker X gets **no brokerage**: B's 80 profit outweighs A's loss once they're combined. The two sheets answer different questions, so their totals differ. Summary: 3 brokers, 4 clients, 1 broker in loss, total brokerage 20.00.

Part 3. Build it step by step

Step 1. Constants and imports

What I was thinking
"Names used in several functions go at the top, in capitals, so they're defined once and easy to find."
"LEDGER_COLUMNS is a set because I only ever ask 'which are missing?', and set subtraction answers that."
"Import the shared helpers from core's pure modules, so this file still needs no Django."

```
from __future__ import annotations

import numpy as np
import pandas as pd

from core.periods import month_range          # shared date helper (plain pandas)
from core.readers import read_csv             # shared CSV reader (plain pandas)

TRADING_TYPES = ["BILL", "NEGATIVE BALANCE PROTECTION"]
BROKER_COLUMNS = ["Broker Name", "Broker Client ID", "Broker Referral Count", "Referred User Client ID"]
LEDGER_COLUMNS = {"ClientID", "CreatedAt", "LedgerType", "Debit", "Credit"}
```

Tool	What it does	Why here
`from __future__ import annotations`	Type hints aren't evaluated at import	Newer hint syntax works on older Pythons
`UPPER_CASE names`	A convention for constants	Signals 'don't change at runtime'
`[...] vs {...}`	List keeps order; set is for membership	BROKER_COLUMNS also orders the output columns
`from core.periods import month_range`	Reuses the shared helper	One definition of a month everywhere

Check yourself
python -c "import commission.services" from the project folder works without Django settings.

Step 2. read_brokers: a clean broker table

What I was thinking
"core.read_csv already checks columns and strips text; I only add what's specific to this file."
"Broker Referral Count arrives as text. to_numeric with errors='coerce' turns 'six' or blank into NaN instead of crashing, then fillna(0).astype(int) makes it a whole number."
"A row without a referred client ID can't be matched to anything, so drop it."

```
def read_brokers(file) -> pd.DataFrame:
    """Read the broker referrals CSV (a path or an uploaded file) and keep the four columns we need."""
    df = read_csv(file, required=BROKER_COLUMNS,
                  text_columns=["Broker Name", "Broker Client ID", "Referred User Client ID"])[BROKER_COLUMNS].copy()
    df["Broker Referral Count"] = pd.to_numeric(df["Broker Referral Count"], errors="coerce").fillna(0).astype(int)
    return df.dropna(subset=["Referred User Client ID"])
```

Tool	What it does	Why here
`read_csv(..., required=, text_columns=)`	Reads, checks and strips	Clear errors come for free
`[BROKER_COLUMNS].copy()`	Keeps only these columns, as a new table	Extra export columns are dropped
`pd.to_numeric(s, errors='coerce')`	Text to numbers; bad values become NaN	No crash on 'six' or blanks
`.fillna(0).astype(int)`	NaN to 0, then whole numbers	6, not 6.0
`df.dropna(subset=[col])`	Drops rows blank in that column	Unmatchable rows go

Check yourself
A row with ' A ' as the referred ID comes back as 'A'; a row with no referred ID is dropped.

Step 3. client_pnl, part A: choose the rows

What I was thinking
"Fail early if the ledger is missing a column, with a message, not a KeyError later."
"Build one True/False mask from four conditions: in the month, a referred client, and a trading type."
"Clean the ledger type once and keep it beside the rows: filtering both with the same mask keeps them aligned by index."

```
def client_pnl(led: pd.DataFrame, clients, start, end, rate: float = 0.05) -> pd.DataFrame:
    """Loss, profit, NBP, net profit and commission per client, for start <= CreatedAt < end."""
    missing = LEDGER_COLUMNS - set(led.columns)
    if missing:
        raise ValueError(f"The ledger is missing columns: {', '.join(sorted(missing))}")

    created = pd.to_datetime(led["CreatedAt"])
    ledger_type = led["LedgerType"].astype(str).str.strip().str.upper()
    keep = (created >= start) & (created < end) & led["ClientID"].isin(set(clients)) & ledger_type.isin(TRADING_TYPES)

    rows, lt = led[keep], ledger_type[keep]
```

Tool	What it does	Why here
`LEDGER_COLUMNS - set(led.columns)`	Required columns that aren't there	A clear error before any work
`pd.to_datetime(...)`	Makes sure CreatedAt is a date	Safe even if it already is one
`.astype(str).str.strip().str.upper()`	Text, no spaces, capitals	'bill ' counts as BILL
`a & b & c & d`	All four must be true	One mask, built from Series of the same index
`.isin(set(clients))`	Membership test	A set makes the lookup fast
`led[keep], ledger_type[keep]`	The same rows of both	It stays aligned with rows by index

Check yourself
In the worked example, B's 1 Sep row and all of C's (none) are excluded; A's 'bill ' row is kept.

Step 4. client_pnl, part B: totals per client

What I was thinking
"Each amount is a column that's the value on the right rows and 0 elsewhere: then one groupby sum does all three."
"Net and Commission depend on the summed columns, so they're added after the groupby. lambda x: lets assign use the table as it is at that point in the chain."
"Commission is a percentage of a loss only: np.where picks it or 0 per row."

```
    debit, credit = rows["Debit"].fillna(0), rows["Credit"].fillna(0)

    return (rows.assign(Loss=debit.where(lt.eq("BILL"), 0),
                        Profit=credit.where(lt.eq("BILL"), 0),
                        NBP=credit.where(lt.eq("NEGATIVE BALANCE PROTECTION"), 0))
                .groupby("ClientID", as_index=False)[["Loss", "Profit", "NBP"]].sum()
                .assign(Net_Profit=lambda x: x["Profit"] - x["Loss"] + x["NBP"],
                        Commission=lambda x: np.where(x["Net_Profit"] < 0, -x["Net_Profit"] * rate, 0.0)))
```

Tool	What it does	Why here
`s.fillna(0)`	Blank amounts become 0	One blank can't make a total blank
`debit.where(lt.eq('BILL'), 0)`	Keep where true, 0 elsewhere	The loss column, built in one line
`groupby(..., as_index=False)[cols].sum()`	One row per client, ClientID as a column	Ready to merge later
`.assign(new=lambda x: ...)`	x is the table at this point in the chain	Uses the summed columns
`np.where(cond, a, b)`	a where true, b where false	Commission only on a net loss
`-x['Net_Profit'] * rate`	Turns a loss into a positive amount	-(-60) x 0.05 = 3.0

Check yourself
Client A: Loss 100, Profit 30, NBP 10, Net -60, Commission 3.0.

Step 5. client_sheet: every referred client, traded or not

What I was thinking
"The broker file is the list of who should appear. pnl only has clients who traded. A left merge from brokers keeps everyone; the missing values become 0."
"Return the columns in a fixed order, and drop the duplicate ClientID column the merge added."

```
def client_sheet(brokers: pd.DataFrame, pnl: pd.DataFrame) -> pd.DataFrame:
    """One row per referred client, with that client's P&L (zeros if they didn't trade)."""
    money = ["Loss", "Profit", "NBP", "Net_Profit", "Commission"]
    out = brokers.merge(pnl, left_on="Referred User Client ID", right_on="ClientID", how="left")
    out[money] = out[money].fillna(0)
    return out[BROKER_COLUMNS + money]
```

Tool	What it does	Why here
`merge(left_on=, right_on=, how='left')`	Joins on differently named keys, keeping every left row	Clients with no trades stay
`out[money] = out[money].fillna(0)`	Fills several columns at once	No trades means 0, not blank
`out[BROKER_COLUMNS + money]`	Selects columns in this order	List + list joins them; ClientID is dropped

Check yourself
Client C appears with zeros in all five money columns.

Step 6. broker_sheet, part A: totals per broker

What I was thinking
"One row per broker with sums, a count and the list of children. agg with name=(column, function) pairs does it in one call."
"The output names have spaces and brackets, which aren't valid keyword names, so pass them through a dict with **."
"No built-in joins text per group, so this is the right place for a lambda."

```
def broker_sheet(clients: pd.DataFrame, rate: float = 0.05) -> pd.DataFrame:
    out = (clients.groupby("Broker Client ID", as_index=False)
                  .agg(**{"Broker Name": ("Broker Name", "first"),
                          "Children": ("Referred User Client ID", lambda s: ",".join(s.astype(str))),
                          "No of Children": ("Referred User Client ID", "nunique"),
                          "Total Profit(Credit)": ("Profit", "sum"),
                          "Total Loss(Debit)": ("Loss", "sum"),
                          "Total Negative Balance Protection": ("NBP", "sum")})
                  .rename(columns={"Broker Client ID": "Parent"}))
```

Tool	What it does	Why here
`.agg(name=(column, function))`	Named aggregation	Each output column says where it comes from
`.agg(**{'Name with spaces': ...})`	Unpacks a dict into keyword arguments	Names that aren't valid Python names
`'first'`	The first value in each group	The broker's name
`lambda s: ','.join(s.astype(str))`	Joins one group's values into text	'A,B' in the Children column
`'nunique'`	Distinct values per group	A client listed twice counts once
`.rename(columns={...})`	Renames columns	The MIS file calls it Parent

Check yourself
Broker X: Children 'A,B', No of Children 2, Profit 110, Loss 100, NBP 10.

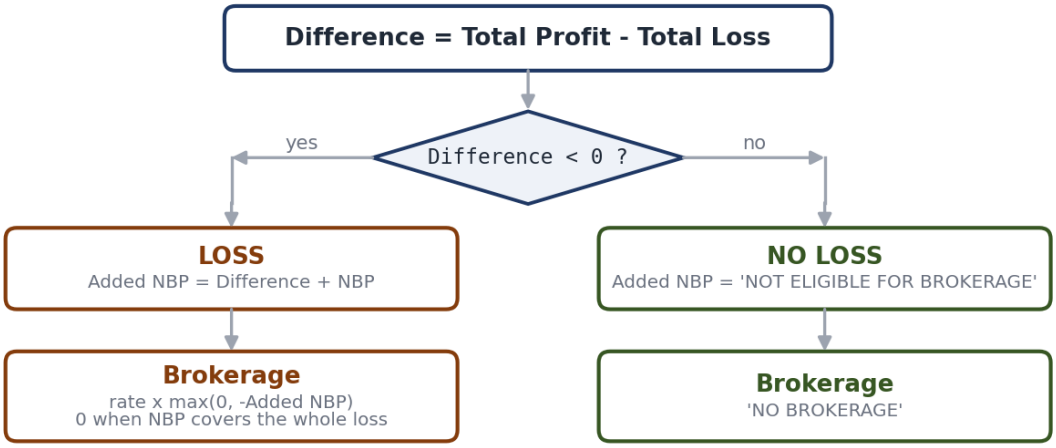
Step 7. broker_sheet, part B: the MIS rules

What I was thinking
"Compute the pieces once and name them: is_loss and added_nbp are used in three lines below."
"Loss Status is always text, so np.where is fine. Added NBP and Brokerage mix numbers with text: np.where would turn every number into text, so use Series.where, which keeps the numbers numeric."
"If NBP covers the whole loss, added_nbp turns positive. Flip the sign and clip at 0, so a covered loss pays 0, never a commission on a gain."

```
    out["Difference"] = out["Total Profit(Credit)"] - out["Total Loss(Debit)"]
    is_loss = out["Difference"] < 0
    added_nbp = out["Difference"] + out["Total Negative Balance Protection"]

    out["Loss Status"] = np.where(is_loss, "LOSS", "NO LOSS")
    out["Added NBP"] = added_nbp.where(is_loss, "NOT ELIGIBLE FOR BROKERAGE")
    out["Brokerage"] = ((-added_nbp).clip(lower=0) * rate).where(is_loss, "NO BROKERAGE")

    return out[["Broker Name", "Parent", "Children", "No of Children",
                "Total Profit(Credit)", "Total Loss(Debit)", "Difference",
                "Total Negative Balance Protection", "Loss Status", "Added NBP", "Brokerage"]]
```



`Example from the MIS file:  -201,581.25 + 5,889.27 = -195,691.98;  5% of 195,691.98 = 9,784.60`

The rule runs on the broker's combined totals, so one client's profit can offset another's loss.

Tool	What it does	Why here
`is_loss = out['Difference'] < 0`	A True/False Series	Named once, used three times
`np.where(is_loss, 'LOSS', 'NO LOSS')`	Two labels	Both sides are text: np.where is fine
`s.where(cond, 'TEXT')`	Keep s where true, the text elsewhere	Numbers stay numbers in a mixed column
`(-added_nbp).clip(lower=0)`	Loss as a positive amount, floored at 0	A loss covered by NBP pays nothing
`out[[...]]`	The final column order	Matches the MIS layout

Check yourself
A broker with Difference -50 and NBP 80: LOSS, Added NBP 30.0, Brokerage 0.0.

Step 8. build_commission: the whole report

What I was thinking
"The view should make one call. This function runs the steps in order and returns everything the page and the Excel need."
"The Brokerage column mixes numbers and text. to_numeric(errors='coerce') turns the text into NaN, so the sum adds only real brokerage."
"Headline numbers as plain int and float, so they save cleanly into the model."

```
def build_commission(led: pd.DataFrame, brokers: pd.DataFrame, month: str, rate: float = 0.05) -> dict:
    """The whole report for one month: both sheets plus headline numbers."""
    start, end = month_range(month)
    pnl = client_pnl(led, brokers["Referred User Client ID"], start, end, rate)
    clients = client_sheet(brokers, pnl)
    brokers_out = broker_sheet(clients, rate)
    brokerage = pd.to_numeric(brokers_out["Brokerage"], errors="coerce").fillna(0)
    return {
        "clients": clients,
        "brokers": brokers_out,
        "summary": {
            "brokers": int(len(brokers_out)),
            "clients": int(clients["Referred User Client ID"].nunique()),
            "loss_brokers": int(brokers_out["Loss Status"].eq("LOSS").sum()),
            "total_brokerage": round(float(brokerage.sum()), 2),
        },
    }
```

Tool	What it does	Why here
`start, end = month_range(month)`	Unpacks the tuple	The same month rule as every report
`pd.to_numeric(..., errors='coerce')`	'NO BROKERAGE' becomes NaN	Only numbers are summed
`.eq('LOSS').sum()`	Counts True values	Brokers in loss
`int(...), round(float(...), 2)`	Plain Python numbers	Save cleanly into model fields
`return {...}`	A dict of results	The view reads report['brokers'], report['summary']

Check yourself
The worked example returns summary {'brokers': 3, 'clients': 4, 'loss_brokers': 1, 'total_brokerage': 20.0}.

Part 4. Mistakes this file prevents

If you wrote...	What would happen	What the code does instead
`led[led['LedgerType'] == 'BILL']`	'bill ' rows silently dropped	strip + upper before comparing
`Filtering BILL first, then BILL + NBP`	NBP always 0 (the 4 Nov bug)	Both types filtered in one mask
`sum() with blank Debit values`	A row's subtraction becomes NaN	fillna(0) on Debit and Credit
`pnl.merge(brokers) (pnl on the left)`	Clients with no trades disappear	brokers.merge(pnl, how='left')
`np.where(is_loss, added_nbp, 'TEXT')`	Every number turned into text	added_nbp.where(is_loss, 'TEXT')
`abs(added_nbp) * rate`	Brokerage paid on a gain when NBP covers the loss	(-added_nbp).clip(lower=0)
`brokers_out['Brokerage'].sum()`	TypeError: number + text	to_numeric(errors='coerce') first
`Summing client Commission for brokers`	Different from the MIS rule	broker_sheet applies the rule to totals
`import from django in this file`	Can't test or reuse it as plain pandas	Only core.periods and core.readers

Part 5. Checkpoints

These were extracted from this guide and run against the project; they pass.

```
from io import BytesIO

import pandas as pd
from django.test import SimpleTestCase

from commission import services


class CommissionCheckpoints(SimpleTestCase):
    def setUp(self):
        self.led = pd.DataFrame({
            "ClientID": ["A", "A", "A", "B", "B", "D"],
            "CreatedAt": pd.to_datetime(["2026-08-02 09:00", "2026-08-03 09:00", "2026-08-04 09:00",
                                         "2026-08-05 09:00", "2026-09-01 09:00", "2026-08-06 09:00"]),
            "LedgerType": ["BILL", "bill ", "NEGATIVE BALANCE PROTECTION", "BILL", "BILL", "BILL"],
            "Debit": [100.0, None, 0.0, 0.0, 500.0, 400.0],
            "Credit": [0.0, 30.0, 10.0, 80.0, 0.0, 0.0]})
        self.brokers = pd.DataFrame({"Broker Name": ["X", "X", "Y", "Z"], "Broker Client ID": ["P1", "P1", "P2", "P3"],
                                     "Broker Referral Count": [2, 2, 1, 1],
                                     "Referred User Client ID": ["A", "B", "C", "D"]})

    def test_worked_example(self):
        r = services.build_commission(self.led, self.brokers, "2026-08")
        clients = r["clients"].set_index("Referred User Client ID")
        brokers = r["brokers"].set_index("Parent")
        self.assertEqual(clients.loc["A", "Commission"], 3.0)
        self.assertEqual(clients.loc["C", "Loss"], 0)
        self.assertEqual(brokers.loc["P1", "Brokerage"], "NO BROKERAGE")      # B's profit offsets A's loss
        self.assertEqual(brokers.loc["P3", "Brokerage"], 20.0)
        self.assertEqual(r["summary"], {"brokers": 3, "clients": 4, "loss_brokers": 1, "total_brokerage": 20.0})

    def test_nbp_covering_the_loss_pays_nothing(self):
        led = pd.DataFrame({"ClientID": ["Q", "Q"], "CreatedAt": pd.to_datetime(["2026-08-02 10:00", "2026-08-03 10:00"]),
                            "LedgerType": ["BILL", "NEGATIVE BALANCE PROTECTION"], "Debit": [50.0, 0.0], "Credit": [0.0, 80.0]})
        b = pd.DataFrame({"Broker Name": ["W"], "Broker Client ID": ["P9"], "Broker Referral Count": [1],
                          "Referred User Client ID": ["Q"]})
        row = services.build_commission(led, b, "2026-08")["brokers"].iloc[0]
        self.assertEqual((row["Loss Status"], row["Added NBP"], row["Brokerage"]), ("LOSS", 30.0, 0.0))

    def test_read_brokers_cleans(self):
        csv = b"Broker Name,Broker Client ID,Broker Referral Count,Referred User Client ID\n X ,P1,6, A \nX,P1,6,\n"
        df = services.read_brokers(BytesIO(csv))
        self.assertEqual(df["Referred User Client ID"].tolist(), ["A"])
        self.assertEqual(df["Broker Referral Count"].tolist(), [6])
```

Explain it back

Answer each in one or two sentences, without looking. The answers follow.

1. Why is LEDGER_COLUMNS a set and BROKER_COLUMNS a list?
2. Why are ledger_type and rows filtered with the same mask?
3. Why does client_pnl fill blank Debit and Credit with 0 before summing?
4. Why does client_sheet merge brokers onto pnl, not pnl onto brokers?
5. Why Series.where for Added NBP and Brokerage, but np.where for Loss Status?
6. What does (-added_nbp).clip(lower=0) protect against?
7. In the worked example, why does broker X get no brokerage although client A has a commission?
8. Why does build_commission use pd.to_numeric(errors='coerce') before summing Brokerage?

Answers

1. LEDGER_COLUMNS is only used for 'which are missing?' (set subtraction); BROKER_COLUMNS also fixes the output order.
2. So the cleaned types stay aligned, row for row, with the rows they describe (same index).
3. Each row's Withdraw-style arithmetic would turn NaN, and sum would silently skip that row.
4. The broker file is the full list; a left merge from it keeps clients with no trades, with zeros.
5. Those two mix numbers and text; np.where would make every value text. Loss Status is text on both sides.
6. Paying brokerage on a gain when NBP is larger than the loss.
7. The rule runs on the broker's combined totals: B's 80 profit plus A's 30 outweigh A's 100 loss.
8. The column mixes numbers with 'NO BROKERAGE'; coercing turns the text into NaN, so only numbers are added.

Appendix. The file, with line numbers

commission/services.py

```
  1  """
  2  Commission calculation: plain pandas, no Django imports.
  3
  4  Keeping the business logic here (not in views.py) means it can be tested on tiny tables,
  5  reused in a notebook, and changed without touching the web parts.
  6  """
  7  from __future__ import annotations
  8
  9  import numpy as np
 10  import pandas as pd
 11
 12  from core.periods import month_range          # shared date helper (plain pandas)
 13  from core.readers import read_csv             # shared CSV reader (plain pandas)
 14
 15  TRADING_TYPES = ["BILL", "NEGATIVE BALANCE PROTECTION"]
 16  BROKER_COLUMNS = ["Broker Name", "Broker Client ID", "Broker Referral Count", "Referred User Client ID"]
 17  LEDGER_COLUMNS = {"ClientID", "CreatedAt", "LedgerType", "Debit", "Credit"}
 18
 19
 20  def read_brokers(file) -> pd.DataFrame:
 21      """Read the broker referrals CSV (a path or an uploaded file) and keep the four columns we need."""
 22      df = read_csv(file, required=BROKER_COLUMNS,
 23                    text_columns=["Broker Name", "Broker Client ID", "Referred User Client ID"])[BROKER_COLUMNS].copy()
 24      df["Broker Referral Count"] = pd.to_numeric(df["Broker Referral Count"], errors="coerce").fillna(0).astype(int)
 25      return df.dropna(subset=["Referred User Client ID"])
 26
 27
 28  def client_pnl(led: pd.DataFrame, clients, start, end, rate: float = 0.05) -> pd.DataFrame:
 29      """Loss, profit, NBP, net profit and commission per client, for start <= CreatedAt < end."""
 30      missing = LEDGER_COLUMNS - set(led.columns)
 31      if missing:
 32          raise ValueError(f"The ledger is missing columns: {', '.join(sorted(missing))}")
 33
 34      created = pd.to_datetime(led["CreatedAt"])
 35      ledger_type = led["LedgerType"].astype(str).str.strip().str.upper()
 36      keep = (created >= start) & (created < end) & led["ClientID"].isin(set(clients)) & ledger_type.isin(TRADING_TYPES)
 37
 38      rows, lt = led[keep], ledger_type[keep]
 39      debit, credit = rows["Debit"].fillna(0), rows["Credit"].fillna(0)
 40
 41      return (rows.assign(Loss=debit.where(lt.eq("BILL"), 0),
 42                          Profit=credit.where(lt.eq("BILL"), 0),
 43                          NBP=credit.where(lt.eq("NEGATIVE BALANCE PROTECTION"), 0))
 44                  .groupby("ClientID", as_index=False)[["Loss", "Profit", "NBP"]].sum()
 45                  .assign(Net_Profit=lambda x: x["Profit"] - x["Loss"] + x["NBP"],
 46                          Commission=lambda x: np.where(x["Net_Profit"] < 0, -x["Net_Profit"] * rate, 0.0)))
 47
 48
 49  def client_sheet(brokers: pd.DataFrame, pnl: pd.DataFrame) -> pd.DataFrame:
 50      """One row per referred client, with that client's P&L (zeros if they didn't trade)."""
 51      money = ["Loss", "Profit", "NBP", "Net_Profit", "Commission"]
 52      out = brokers.merge(pnl, left_on="Referred User Client ID", right_on="ClientID", how="left")
 53      out[money] = out[money].fillna(0)
 54      return out[BROKER_COLUMNS + money]
 55
 56
 57  def broker_sheet(clients: pd.DataFrame, rate: float = 0.05) -> pd.DataFrame:
 58      """One row per broker, with the MIS rules: brokerage on the broker's combined loss after NBP."""
 59      out = (clients.groupby("Broker Client ID", as_index=False)
 60                    .agg(**{"Broker Name": ("Broker Name", "first"),
 61                            "Children": ("Referred User Client ID", lambda s: ",".join(s.astype(str))),
 62                            "No of Children": ("Referred User Client ID", "nunique"),
```

```
                           "Total Profit(Credit)": ("Profit", "sum"),
                           "Total Loss(Debit)": ("Loss", "sum"),
                           "Total Negative Balance Protection": ("NBP", "sum")})
                   .rename(columns={"Broker Client ID": "Parent"}))

    out["Difference"] = out["Total Profit(Credit)"] - out["Total Loss(Debit)"]
    is_loss = out["Difference"] < 0
    added_nbp = out["Difference"] + out["Total Negative Balance Protection"]

    out["Loss Status"] = np.where(is_loss, "LOSS", "NO LOSS")
    out["Added NBP"] = added_nbp.where(is_loss, "NOT ELIGIBLE FOR BROKERAGE")
    out["Brokerage"] = ((-added_nbp).clip(lower=0) * rate).where(is_loss, "NO BROKERAGE")

    return out[["Broker Name", "Parent", "Children", "No of Children",
                "Total Profit(Credit)", "Total Loss(Debit)", "Difference",
                "Total Negative Balance Protection", "Loss Status", "Added NBP", "Brokerage"]]


def build_commission(led: pd.DataFrame, brokers: pd.DataFrame, month: str, rate: float = 0.05) -> dict:
    """The whole report for one month: both sheets plus headline numbers."""
    start, end = month_range(month)
    pnl = client_pnl(led, brokers["Referred User Client ID"], start, end, rate)
    clients = client_sheet(brokers, pnl)
    brokers_out = broker_sheet(clients, rate)
    brokerage = pd.to_numeric(brokers_out["Brokerage"], errors="coerce").fillna(0)
    return {
        "clients": clients,
        "brokers": brokers_out,
        "summary": {
            "brokers": int(len(brokers_out)),
            "clients": int(clients["Referred User Client ID"].nunique()),
            "loss_brokers": int(brokers_out["Loss Status"].eq("LOSS").sum()),
            "total_brokerage": round(float(brokerage.sum()), 2),
        },
    }
```

Lines	Block	Purpose
7-17	`Imports + constants`	Shared helpers and fixed names
20-25	`read_brokers()`	Clean broker table
28-38	`client_pnl(): rows`	Check columns, build the mask
39-46	`client_pnl(): totals`	Amount columns, groupby, rules
49-54	`client_sheet()`	Every referred client
57-66	`broker_sheet(): totals`	Named aggregation
68-78	`broker_sheet(): rules`	LOSS, Added NBP, Brokerage
81-97	`build_commission()`	The pipeline and summary